



Panel

Proyectos

Features

Stories

Product Backlog

Sprints

Lanzamientos

Incidentes

Mejoras

Documentación

Change Requests

Reportes

[Panel](#) / [Documentación](#) / Documentación funcional

Documentación

Documentación funcional

[Documentación técnica](#)

Documentos

1. Introducción

Qué es Nexus DevSuite

Nexus DevSuite es una plataforma de gestión de proyectos y entregas de código que une planificación de versiones (releases), backlog SCRUM (features, user stories, sprints), órdenes de trabajo (work orders) y un flujo Git completo: commit, push y Pull Request desde la aplicación, con trazabilidad entre planificación y repositorio.

Qué problema resuelve

- Trazabilidad: vincular releases, features, stories, work orders y entregas de código con ramas, commits y PRs en GitHub.
- Un solo lugar para planificar, desarrollar y entregar sin cambiar de herramienta para el flujo básico de commit/PR.
- Validaciones: snapshot de la entrega, detección de drift (cambios externos), dry run antes de commit, estados (DRAFT, READY, LOCKED) para controlar qué se puede editar o commitear.

Casos de uso reales

- Equipo que planifica releases por features y quiere que cada entrega de código quede ligada a una task y a una rama/PR.
- Desarrollador que sube cambios desde Nexus (sync desde Git o método manual) y hace commit y PR sin usar la consola local.
- Lead que revisa el estado de entregas (COMMITTED, PR_CREATED, MERGED) y genera release notes comparando entregas con IA.

Cuándo usarlo y cuándo no

Usar Nexus DevSuite cuando quieras planificación de releases + backlog SCRUM + flujo de entregas trazado a Git/PR en una sola app. **No usarlo** como sustituto de Git en tu máquina para trabajo local diario (branching, merge, rebase); úsalo como orquestador de entregas y PRs a partir del código que ya tienes en el servidor o que subes manualmente.

2. Quick Start — Primera entrega en menos de 10 pasos

Flujo mínimo para crear una entrega, subir archivos, hacer commit y crear PR.

Paso 1: Proyectos → abre tu proyecto → Órdenes de trabajo.

Paso 2: Crear orden de trabajo (asocia User Story). En el detalle, añade una Task (Add Task).

Paso 3: En **Entregas de código** → **Crear entrega**. Task, título, tipo (Feature/Bugfix). Guardar.

Paso 4: Ir a **Repository** del proyecto. En la fila de tu entrega → **Crear rama**.

Paso 5: En la misma tabla → **Workspace** (abre el Delivery Workspace).

Paso 6: En "Subir entrega (Git como fuente de verdad)" → **Detectar archivos** → **Subir entrega desde Git**. (O método manual: pegar `git status --short`, Extraer archivos, elegir carpeta, Subir seleccionados.)

Paso 7: En la tarjeta **Files**, deja marcados los archivos que quieras en el commit. Cambia el estado de la entrega a **READY** (o LOCKED) si aún no lo está.

Paso 8: En **Commit & Push**: escribe mensaje (o **Generar con IA**), marca **Crear Pull Request después del push** si quieres PR.

Paso 9: Vista previa → revisar lista y mensaje → **Confirmar commit y push**.

Paso 10: Tras el push, si creaste PR abre el enlace en GitHub para revisión y merge.

3. Workflow completo (paso a paso)

Cada paso con acción en UI, qué ocurre en backend y resultado esperado.

PASO 1 — Crear entrega

Acción: Órdenes de trabajo → detalle WO → Entregas de código → Crear entrega → Task, título, tipo → Guardar.

Backend: POST a code-deliveries; se crea registro vinculado a task_id y project_id; estado inicial PREPARING/DRAFT.

Resultado: La entrega aparece en la lista de entregas de la WO y en Repository del proyecto.

PASO 2 — Crear rama

Acción: Repository del proyecto → fila de la entrega → Crear rama.

Backend: Llamada a GitHub API para crear la rama en el repo (ej. feature/TASK-1-nombre).

Resultado: La entrega muestra el nombre de la rama; sin rama no se puede hacer commit.

PASO 3 — Subir archivos

Acción: Workspace → Detectar archivos (Git) o pegar git status y Subir seleccionados (manual).

Backend: Git: `git status --porcelain=v2` y diff; o recepción de archivos por multipart. Se guardan en delivery_files; opcionalmente se crea snapshot (git_snapshot_json).

Resultado: Tarjeta Files con lista de archivos (ADDED, MODIFIED, DELETED, etc.).

PASO 4 — Validar

Acción: Revisar archivos en Files; poner estado de la entrega en READY o LOCKED.

Backend: Solo con estado READY o LOCKED el commit está permitido (DELIVERY_STATUS_NOT_COMMITTABLE si no).

Resultado: Botones Vista previa y Commit & Push habilitados.

PASO 5 — Commit & Push

Acción: Mensaje de commit, opción Crear PR → Vista previa → Confirmar commit y push.

Backend: compareSnapshotWithCurrent (detecta drift); si OK, commit en repo, push a origin, actualización de commit_hash y status (COMMITTED o PR_CREATED); si create_pr, creación de PR en GitHub.

Resultado: Commit en la rama; enlace al PR si se creó; estado COMMITTED/PR_CREATED.

4. Gestión de archivos

A. Desde Git (recomendado)

El servidor ejecuta Git en el repositorio del proyecto. Comandos que Nexus usa internamente (o que tú puedes usar para comparar):

```
git status --porcelain=v2
```

Salida en formato v2: una línea por archivo con estado (XY), path, y para renames/copies path origen y destino. Nexus parsea esto con `parsePorcelainV2` y obtiene ADDED, MODIFIED, DELETED, RENAMED, COPIED, UNTRACKED.

```
git add .
git diff --cached --name-status
```

Si eliges **Solo staged**, Nexus usa el diff en staging para saber qué archivos incluir. `git add .` prepara todos los cambios; `git diff --cached --name-status` lista los preparados.

```
git diff -- path/to/file.js
```

Para ver el diff de un archivo concreto; Nexus usa el diff para rellenar el contenido que sube y para el visor de diferencias en el Workspace.

En la UI: Tarjeta "Subir entrega (Git como fuente de verdad)" → Incluir working directory o Solo staged → **Detectar archivos** → **Subir entrega desde Git**. El backend construye un mapa unificado por path y sube contenido.

B. Método manual

Cuándo usarlo: Cuando el servidor no tiene el repositorio Git clonado o no puedes usar sync-from-git (ej. desarrollo local en otra máquina).

Riesgos: Rutas o prefijos incorrectos (usar "Prefijo a quitar de rutas" si las rutas incluyen el nombre de la carpeta del proyecto); olvidar archivos; no reflejar exactamente lo que Git vería. En producción se recomienda usar Git como fuente de verdad.

Pasos: Ejecutar `git status` o `git status --short` en tu máquina → copiar toda la salida → Workspace → Método manual → Pegar en Paso 1 → Extraer archivos modificados → Paso 2 elegir carpeta → Subir seleccionados a la entrega.

Estados de archivo (MODIFIED, ADDED, DELETED, RENAMED, COPIED)

Estado	Significado
MODIFIED	Archivo existente con cambios respecto a la base.

ADDEDArchivo nuevo (no existía en la rama base/master).DELETEDArchivo eliminado.RENAMEDArchivo renombrado (origen → destino).COPIEDArchivo copiado (origen → copia).

5. Estados de entrega

Estado	Qué significa	Permite	Bloquea
PREPARING	Inicial; entrega recién creada.	Editar metadatos, añadir archivos.	Commit.
DRAFT	Borrador en edición.	Añadir/quitar archivos, sync desde Git, editar.	Commit (hay que pasar a READY o LOCKED).

Estado	Qué significa	Permite	Bloquea
READY	Validada; lista para commit.	Commit, push, crear PR; modificar archivos y sync.	Nada crítico.
LOCKED	Snapshot congelado.	Commit y push con el contenido actual.	Añadir/quitar archivos, sync-from-git (error DELIVERY_LOCKED).
COMMITTED	Commit y push realizados.	Ver historial; nuevo ciclo requiere nueva entrega o re-apertura según tu proceso.	Editar archivos de esta entrega.
PR_CREATED	Pull Request creado en GitHub.	Enlace al PR; revisión en GitHub.	Editar archivos de esta entrega.
MERGED	PR mergeado en GitHub.	Trazabilidad cerrada.	Editar archivos de esta entrega.

Ejemplo LOCKED: Caso de uso típico: congelar la entrega justo antes de producción; nadie puede añadir o quitar archivos; solo se permite commit con lo ya incluido. Evita cambios de último momento no revisados.

6. Commit desde Nexus (guía operativa)

- **Selección de archivos:** En la tarjeta Files cada archivo tiene un checkbox; solo los marcados se incluyen en el commit. Usa Todos / Ninguno para marcar o desmarcar todos.
- **Preview:** Vista previa llama a la API commit-preview: muestra archivos que se incluirán, resumen de impacto (líneas +/-) y mensaje. Si hay SNAPSHOT_DRIFT no podrás confirmar hasta re-sincronizar.
- **Validaciones:** (1) Estado READY o LOCKED. (2) Si LOCKED, no se pueden cambiar archivos ni sync. (3) Snapshot sin drift (código no cambió fuera de Nexus desde el último snapshot).

Ejemplo real de mensaje de commit

```
feat(auth): implement login validation

Delivery: DEL-001
WO: Fix login issue
```

Formato convencional: tipo(alcance): descripción. Líneas opcionales para referencia a entrega y work order.

7. Validaciones y errores

SNAPSHOT_DRIFT

Qué significa: El contenido del repositorio (o de los archivos de la entrega en BD) no coincide con el snapshot guardado en la entrega (git_snapshot_json).

Por qué ocurre: Alguien modificó archivos en el repo después de que se creó el snapshot; o se editaron archivos de la entrega fuera del flujo de sync; o se borraron/añadieron archivos sin volver a sincronizar.

Solución paso a paso: (1) En el Workspace, vuelve a **Detectar archivos** y **Subir entrega desde Git** para regenerar el snapshot con el estado actual. (2) O revierte los cambios externos en el repo y vuelve a abrir el Workspace. (3) Tras eso, Vista previa debería pasar y podrás Confirmar commit y push.

DELIVERY_LOCKED

Causa: La entrega está en estado LOCKED y se intentó añadir, eliminar o actualizar archivos, o llamar a sync-from-git.

Solución: Si debes cambiar archivos, cambia el estado de la entrega a READY (desde la gestión de la WO/entrega, si tu proceso lo permite). Si la política es no desbloquear, crea una nueva entrega con los cambios.

DELIVERY_STATUS_NOT_COMMITTABLE

Causa: La entrega no está en READY ni LOCKED (está en PREPARING, DRAFT, etc.) y se intentó hacer commit o preview de commit.

Solución: Cambiar el estado de la entrega a **READY** o **LOCKED** desde la interfaz (detalle de la entrega o de la orden de trabajo). Solo entonces estarán habilitados Vista previa y Commit & Push.

8. Dry run y seguridad

Qué es simulateCommit: Es un dry run del commit: el backend valida los mismos pasos que en el commit real (estado READY/LOCKED, snapshot sin drift, archivos seleccionados) pero **no escribe** en el repositorio Git. La API es `POST .../commit-simulate` con el mismo payload que commit-preview.

Cuándo usarlo: Antes de Commit & Push en entornos críticos; para comprobar que no hay SNAPSHOT_DRIFT ni otros fallos sin realizar el commit.

Cómo interpretarlo: Si la respuesta es correcta (ok, validations), el commit real con el mismo payload debería funcionar. Si devuelve errores (snapshot, branch, files), corrige lo indicado antes de hacer el commit real.

9. IA en Nexus

Generación de mensaje de commit: El botón **Generar con IA** llama al endpoint suggest-commit-message. La IA recibe información sobre los archivos de la entrega y el tipo (Feature, Bugfix, etc.) y propone un mensaje en formato Conventional Commits (feat, fix, docs, chore, etc.).

Cómo mejorar resultados: Incluir en la entrega solo los archivos que corresponden al cambio que quieres describir; elegir el tipo de entrega correcto (Feature/Bugfix); editar la sugerencia para añadir alcance o referencia (Delivery, WO).

Limitaciones: La IA no conoce el contexto completo del negocio; puede generalizar. Revisa siempre el mensaje antes de confirmar. La generación depende del servicio de IA configurado en el backend.

10. Release notes (pro)

Se generan comparando la entrega actual con una **entrega base** que eliges en el desplegable (otherDeliveryId). El backend calcula diff (added, removed, modified) y envía un resumen a la IA para producir bullets (Features/changes, Fixes).

Ejemplo de salida

```
## Features / changes
- Add login validation and error messages
- Update auth middleware
```


- ## Fixes
- Fix redirect loop on session expiry

Estructura recomendada: Usar como base la última entrega mergeada de la rama principal o la entrega del tag anterior; así las release notes describen solo lo nuevo de esta entrega.

11. Flujo SCRUM completo (guía operativa)

Sprint típico

1. Crear release (Release Planning → Create Release → versión, nombre).
2. Crear features y asociarlas a la release (Add en Available Features).
3. Crear user stories (Backlog) asociadas a esas features; poner en READY.
4. Asignar stories al sprint (Sprints → sprint activo → asignar).
5. Crear work orders por story; en cada WO, crear tasks y luego **Crear entrega** por task.
6. Repository → Crear rama por entrega → Workspace → subir archivos → READY/LOCKED → Commit & Push → Crear PR.

Al mergear los PRs en GitHub, las entregas pasan a MERGED y el resumen técnico de la release (entregas, PRs, commits) queda actualizado.

12. Mejores prácticas

- **Usar LOCKED antes de commit en producción:** Congela el conjunto de archivos para que nadie añada cambios de último momento sin revisión.
- **Siempre usar Vista previa:** Evita sorpresas por SNAPSHOT_DRIFT o archivos no deseados; revisa el resumen antes de confirmar.
- **Git como fuente de verdad:** Preferir Detectar archivos + Subir desde Git frente al método manual; así el estado en Nexus coincide con el del repositorio.
- **Evitar método manual en producción:** Riesgo de rutas incorrectas o archivos faltantes; usar solo cuando el servidor no tenga el repo o sea un caso excepcional.

13. Troubleshooting

No detecta archivos

Síntoma: Detectar archivos no muestra nada o muestra lista vacía.

Pasos: (1) Comprobar que el proyecto tiene repositorio configurado y que el servidor tiene acceso al repo clonado. (2) Si usas "Solo staged", asegurarte de que hay archivos en staging (`git diff --cached --name-only`). (3) Si el repo está en otra ruta o branch, verificar configuración del proyecto. (4) Como alternativa, usar método manual con la salida de `git status --short` desde tu máquina.

Commit falla

Síntoma: Al confirmar Commit & Push aparece error o no se completa.

Pasos: (1) Revisar mensaje de error: si es SNAPSHOT_DRIFT, re-sincronizar (Detectar archivos → Subir desde Git) y volver a Vista previa. (2) Si es DELIVERY_STATUS_NOT_COMMITTABLE, cambiar estado a READY o LOCKED. (3) Si es DELIVERY_LOCKED y necesitas cambiar archivos, cambiar estado a READY si la política lo permite. (4) Comprobar que la rama existe en GitHub (Crear rama desde Repository si falta). (5) Revisar logs del backend para errores de Git o de GitHub API.

PR no se crea



API: OK